

Refinamento e Implementação Assistidos por Agentes: um Estudo de Caso sobre uma Ferramenta de Edição de Textos em Latex

Anonymous Author(s)

Resumo

A separação entre refinamento de requisitos e implementação de código orienta a proposta deste artigo, apoiada por dois agentes de Inteligência Artificial (IA) especializados: um transforma descrições informais de funcionalidades em prompts estruturados e detalhados, e o outro implementa essas especificações a partir dos prompts recebidos. Como estudo de caso, foi construída uma ferramenta de edição de textos acadêmicos em $\text{\LaTeX}$, cujas funcionalidades nasceram desse processo assistido por agentes. A separação de responsabilidades reduziu ambiguidades nos requisitos e apoiou a implementação das funcionalidades solicitadas. Um profissional técnico, porém, ainda é necessário. Cabe a ele orientar o refinamento e validar os artefatos gerados pelos agentes. O artigo discute benefícios, limitações e implicações dessa estratégia para a Engenharia de Software, além dos desafios de confiabilidade das respostas geradas e de validação das implementações produzidas.

Palavras-chave

Engenharia de Requisitos, LLM, Agentes, Inteligência Artificial, Engenharia de Prompt

1 Introdução

Diante dos recentes avanços de modelos de inteligência artificial (IA), a Engenharia de Software (ES) vem se modificando para se adequar a tais práticas. As ferramentas baseadas em IA têm auxiliado a sociedade em tarefas cotidianas, facilitando a busca por informações. No ambiente corporativo, ela já apresenta papel fundamental no ciclo de desenvolvimento, ajudando na formulação de requisitos, geração de código, testes e documentação.

A Engenharia de Requisitos (ER) e a de Software estão diretamente ligadas na hora da produção de um software [11]. Com a propagação da cultura de metodologias ágeis, o desenvolvedor tem contato direto com o analista de requisitos para a criação de suas tarefas.

Os requisitos apresentam uma descrição dos serviços que o sistema deve fornecer e das restrições sob as quais deve operar [11]. Sua qualidade está diretamente associada ao sucesso na construção de um software, tendo em vista que o programador deve entender do que se trata e traduzir as palavras em código.

Durante a produção de um software, a ER tem como um de seus principais objetivos estabelecer uma comunicação efetiva entre as partes interessadas e a equipe de desenvolvimento, reduzindo ambiguidades e interpretações incorretas dos requisitos [11]. No contexto de agentes baseados em Large Language Models (LLM), esse desafio permanece presente, uma vez que a qualidade da implementação continua dependente da clareza das instruções fornecidas.

De forma mais recente, agentes autônomos de programação têm se mostrado capazes de implementar funcionalidades completas a partir de uma descrição usando linguagem natural, reduzindo assim o esforço manual necessário dos desenvolvedores de software [2].

Apesar dos avanços tecnológicos e da chegada de novos modelos poderosos, ainda é possível observar que a qualidade dos artefatos produzidos por agentes depende diretamente das instruções fornecidas a eles. Requisitos incompletos, ambíguos e com uma estrutura errada podem gerar resultados inconsistentes durante o processo de desenvolvimento, aumentando o retrabalho e o número de interações necessárias para concluir tal tarefa [2]. Por conta disso, é necessário utilizar técnicas avançadas de Engenharia de Prompt e refinamento de requisitos para melhorar a comunicação entre desenvolvedores humanos e agentes baseados em LLM.

Paralelo a isso, observa-se o surgimento de diversas abordagens diferentes para conseguirmos utilizar tais LLMs, dentre elas a utilização de chaining e a programação multiagente se destacam. Tais práticas visam superar as limitações impostas pela ferramenta, como a falta de raciocínio lógico inerente, alucinações e a ausência de informações atualizadas [6].

Este trabalho irá investigar uma abordagem de desenvolvimento orientada por agentes baseada na separação das atividades de refinamento de requisitos e implementação de software. Como base para o estudo, foi desenvolvido um software para auxiliar a produção de textos acadêmicos utilizando $\text{\LaTeX}$, cujas funcionalidades e elementos de interface foram concebidos por meio de um processo de desenvolvimento assistido por agentes. A proposta consiste em utilizar um agente especializado em transformar as descrições e requisitos de funcionalidades escritas por humanos em prompts robustos, que posteriormente serão utilizados como entrada para um agente autônomo de programação responsável pela implementação da solução.

Como objetivo do trabalho, busca-se analisar o potencial dessa estratégia para apoiar o desenvolvimento de software assistido por IA. Serão discutidos seus benefícios, limitações e implicações para a ES. Para isso, foi conduzido um estudo de caso no qual os requisitos da aplicação foram inicialmente definidos por um desenvolvedor humano, refinados por um agente especializado e posteriormente implementados por um agente autônomo de programação.

A organização deste artigo está dividida em 4 seções. A Seção 2 apresenta os conceitos fundamentais relacionados a agentes de software. A Seção 3 descreve a abordagem proposta e o estudo de caso realizado. Por fim, a Seção 4 apresenta as conclusões e as perspectivas para trabalhos futuros.

2 Fundamentação teórica

As Large Language Models (LLMs) são modelos de inteligência artificial (IA) treinados sobre grandes volumes de dados textuais com o objetivo de compreender e gerar linguagem natural [1]. Com base na arquitetura Transformer, esses modelos foram capazes de aprender e se especializar por meio de padrões linguísticos e relações semânticas que lhes permitem realizar tarefas como geração de texto, tradução, respostas a perguntas complexas e produção

de código. Alguns exemplos utilizados atualmente incluem o GPT, Claude, Gemini e Llama [4].

No meio da Engenharia de Software (ES), esses modelos foram introduzidos inicialmente como assistentes para geração de pequenos trechos de código, passando a apoiar diversas atividades ao longo do ciclo de desenvolvimento. Na Engenharia de Requisitos (ER), por sua vez, auxiliam na elicitação, refinamento e validação dos requisitos [8].

Com o avanço da tecnologia, as LLMs começaram a atuar de forma mais autônoma, diretamente do terminal do computador de um desenvolvedor. Desse modo, assumiram papéis especializados para execução das tarefas, colaborando na realização de atividades complexas de desenvolvimento de software [9].

Atualmente, o principal desafio enfrentado ao integrar as LLMs no processo de codificação são as alucinações, sendo uma limitação crítica dos modelos. Como forma de contornar o problema, pode-se usar de estratégias como Retrieval Augmented Generation (RAG), configuração de modelos mais robustos, feedback contínuo humanizado e prompts mais específicos [4].

Dentre os benefícios observados na literatura, observa-se o aumento da produtividade dos desenvolvedores, redução do tempo necessário para execução de tarefas repetitivas, apoio na documentação, auxílio na geração de testes e identificação de problemas e democratização do acesso ao conhecimento técnico [12].

Diante desse cenário de adoção crescente, a Engenharia de Prompt (EP) vem se tornando uma ferramenta primordial para analisar e mitigar as alucinações geradas pelos modelos de inteligência artificial [10]. Ela contribui na criação de um texto elaborado, propondo instruções estruturadas para orientar as LLMs na execução de tarefas específicas. A qualidade e a clareza no prompt estão diretamente ligadas à consistência e à relevância das respostas geradas.

Diversas técnicas têm sido propostas para melhorar o desempenho das respostas [5]. Entre elas destacam-se o *Role Prompting*, no qual o modelo recebe um papel específico a ser desempenhado, como analista de requisitos, PO ou arquiteto de software. O *Chain-of-Thought*, que incentiva a geração de um raciocínio intermediário para analisar e pensar como será feita a resposta final [4]. O *Structured Prompting*, baseado na definição de um formato estruturado de entrada e saída, reduzindo ambiguidade e tornando as respostas mais consistentes [2]. Por último, o *Few-Shot Prompting*, que se utiliza de exemplos para demonstrar o comportamento esperado do modelo [4].

Essas técnicas de prompt costumam ser combinadas com estratégias mais amplas de organização das LLMs, como os sistemas encadeados, chamados na literatura de chaining, que consistem na utilização de vários modelos especializados realizando atividades conjuntas, de forma que atinjam um objetivo comum [4].

Diferente da estratégia de multiagente dinâmico, o chaining não envolve diálogo e refinamento iterativo entre as IAs. Ele opera de forma linear, com auxílio de um humano para realizar a passagem de conhecimento de um elo para o outro [4].

Na Engenharia de Software, essa prática tem se tornado cada vez mais utilizada para distribuir responsabilidades entre especialistas de IA. Sua abordagem busca produzir a divisão de tarefas em partes menores, na busca de resultados mais elaborados e assertivos [3]. Sua utilização em conjunto com LLMs especializadas para refinamento do prompt e codificação pode potencializar os resultados

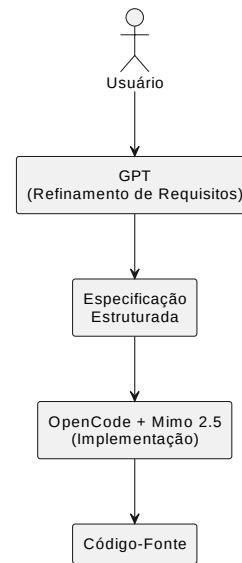


Figure 1: Pipeline de desenvolvimento orientado por agentes para refinamento de requisitos e implementação de software

[9]. Desse modo, é possível produzir um artefato que serve como entrada para agentes responsáveis pela implementação do software.

3 Proposta

A abordagem principal da proposta está composta por 3 etapas, em alusão ao que se observa no mercado de trabalho. Foram utilizados 2 agentes especializados para atuarem como o Engenheiro de Requisitos da aplicação e o Engenheiro de Software, tendo como papel de PO o usuário que está orquestrando-os [9].

O primeiro agente utilizou o modelo GPT-5.5, sendo responsável pelo refinamento e detalhamento dos requisitos passados pelo usuário [8]. Seu objetivo foi transformar descrições informais de funcionalidade em especificações detalhadas e estruturadas no formato de prompts, para serem utilizadas na etapa de desenvolvimento.

O segundo agente foi responsável por pensar na implementação a partir da descrição detalhada dos requisitos. Sua execução foi feita a partir da plataforma visual OpenCode, utilizando o modelo Mimo 2.5. Esse agente, por sua vez, implementou as funcionalidades solicitadas, realizando a criação ou edição do código-fonte da aplicação e produzindo os artefatos necessários para atender aos requisitos definidos.

Desse modo, o fluxo proposto se baseia nas responsabilidades reais do mercado de trabalho, buscando reproduzir a divisão de responsabilidades observada em equipes de desenvolvimento de software. No entanto, faz-se necessário o conhecimento em programação para entender e especificar para a LLM de geração de requisitos a proposta de segurança, as bibliotecas utilizadas e os meios para alcançar tais objetivos.

O agente refinador tem como objetivo transformar solicitações informais e com baixa aceitação pelas LLMs geradoras de código em estruturas mais detalhadas e específicas, auxiliando o desenvolvimento assistido por IA [7]. Para isso, uma descrição textual

elaborada, identificando todos os objetivos de uma funcionalidade, regras de negócio, fluxos principais e critérios de aceite, deve ser passada para o modelo utilizado no refinamento [2]. A LLM é capaz de identificar especificamente pontos de ambiguidade, omissão e inconsistências, aumentando a precisão dos requisitos refinados [7].

Todos os textos devolvidos pelo modelo de refinação devem ser validados e interpretados para garantir o entendimento completo da funcionalidade, evitando que ela seja desenvolvida de forma incompleta ou incorreta [12]. Um ponto de destaque é a importância de existir um profissional técnico por trás da criação desse texto inicial, para conduzir o refinamento orientando as bibliotecas usadas, as validações necessárias e outros pontos que podem impactar o desenvolvimento e a segurança do projeto [2, 7].

Nesse caso empírico, a escolha pela utilização do GPT se deu pela sua alta capacidade criativa e de escrita, se mostrando extremamente habilidoso na estruturação dos prompts [4]. Entretanto, entende-se que qualquer agente capaz de escrever prompts com qualidade pode ser usado nessa etapa de desenvolvimento.

Após o refinamento dos requisitos e especificações de implementação, o prompt gerado deve ser utilizado como entrada para o agente de desenvolvimento, utilizando o modo *thinking* para que ocorra o pensamento e a criação do fluxo das tarefas refinadas pela outra LLM. A partir desse momento, o agente poderá efetuar questionamentos ao desenvolvedor, e esses devem ser analisados e respondidos conforme a necessidade do projeto [4].

Todos os retornos de código devem ser analisados e verificados por um desenvolvedor com capacidade crítica de entender se o que foi feito possui manutenibilidade, confiabilidade e segurança [2, 7]. As implementações devem seguir o padrão no qual seriam feitas pelo próprio desenvolvedor; com isso, ele deve ter total domínio da linguagem de programação utilizada para compreender a estrutura criada.

Um ponto de destaque é a importância de um arquivo com instruções para o agente se tornar mais técnico e específico no projeto, podendo ser um *AGENTS.md* na raiz, especificando toda a arquitetura do projeto para que ele siga os padrões pré-definidos. O guia de arquitetura, funcionalidades, cores e skills utilizadas dentro desse arquivo funcionará como um manual de instruções para ele se basear e entender como deve estruturar as funcionalidades. Contudo, esse manual deve ser novamente refinado com um modelo de inteligência artificial generativo para deixá-lo descritivo e claro.

Dentro do projeto do editor em $\text{\LaTeX}$, foi proposta a utilização da ferramenta open-source OpenCode com o Mímo 2.5, por conta do seu baixo consumo de tokens e da capacidade de pensamento avançado disponível como opção antes da execução do desenvolvimento. Entretanto, é possível se utilizar de outras ferramentas e modelos para essa técnica, tendo em vista a necessidade de um desenvolvedor experiente e com conhecimento na área para tomar a decisão de qual utilizar no projeto.

Para avaliar a aplicabilidade da metodologia proposta, foi desenvolvida uma ferramenta de edição de textos acadêmicos. A ferramenta tem como objetivo auxiliar o processo de escrita, organização e revisão de documentos, integrando recursos tradicionais de edição com funcionalidades baseadas em inteligência artificial. As principais funcionalidades implementadas foram:

- Criação, importação e clonagem de projetos a partir de repositórios Git;
- Listagem e gerenciamento dos projetos criados pelo usuário;
- Interface gráfica para edição de arquivos `.tex`, contendo recursos de leitura e manipulação do código-fonte;
- Criação, edição e organização de subpastas e arquivos dentro do projeto;
- Importação e gerenciamento de imagens utilizadas na composição dos documentos;
- Reorganização de arquivos e diretórios por meio de operações de arrastar e soltar (*drag and drop*);
- Visualização do PDF compilado diretamente na aplicação;
- Navegação por seções do documento, permitindo acesso rápido a capítulos e subseções por meio de âncoras;
- Download do documento PDF gerado;
- Inicialização de repositórios Git nos projetos e execução de comandos básicos de versionamento (em fase de aprimoramento para correção de inconsistências);
- Apresentação de mensagens de erro detalhadas durante o processo de compilação, facilitando a identificação e resolução de problemas;
- Integração com modelos de inteligência artificial para auxílio na revisão textual, incluindo correção ortográfica, análise de concordância verbal, melhoria de sentenças e identificação de repetição excessiva de termos. Foram utilizados modelos locais baseados em Llama e modelos disponibilizados pela plataforma Groq, permitindo que o usuário selecione o provedor desejado por meio de variáveis de ambiente;
- Desenvolvimento da documentação técnica do projeto utilizando a ferramenta MkDocs;
- Implementação de pipeline de integração e entrega contínua (CI/CD) para geração e publicação automática da documentação via GitHub Pages após alterações no repositório principal.

Como trabalhos futuros e itens presentes no backlog de desenvolvimento, destacam-se:

- Implementação de uma análise completa do projeto utilizando inteligência artificial para identificar e sugerir correções ortográficas e gramaticais em todos os arquivos do documento, sem depender do arquivo atualmente aberto pelo usuário;
- Utilização de inteligência artificial para avaliar a aderência do conteúdo desenvolvido aos requisitos definidos pela banca avaliadora, identificando possíveis lacunas ou melhorias necessárias;
- Desenvolvimento de um recurso de assistência ao usuário para esclarecer dúvidas relacionadas à utilização de recursos $\text{\LaTeX}$, incluindo auxílio na aplicação de comandos de formatação textual, como negrito, itálico e outros elementos de estruturação do documento.

A Figura 2 apresenta o layout por dentro da ferramenta desenvolvida para apoiar a elaboração de textos acadêmicos em $\text{\LaTeX}$. A aplicação foi projetada para integrar, em um único ambiente, recursos como organização de documentos e assistência baseada em inteligência artificial.

Na interface, é possível observar a estrutura de arquivos do projeto, permitindo a navegação entre seções, imagens, referências

bibliográficas e demais artefatos que compõem o documento. Foi utilizado o modelo padrão da conferência para exemplificar como seria um texto produzido por ele, além de ser testado de forma prática na construção desse artigo.

A área central é destinada à edição do conteúdo em L^AT_EX, enquanto o painel lateral exibe a visualização do PDF gerado em tempo real, possibilitando ao usuário acompanhar imediatamente os resultados das alterações realizadas.

Além das funcionalidades tradicionais de edição, a ferramenta incorpora recursos voltados a assistentes de IA para apoiar na escrita, refinamento de conhecimento e, em versões futuras, auxílio na utilização de comandos $\LaTeX$. Integrações com plataformas como GitHub também podem ser observadas na imagem, facilitando o gerenciamento e versionamento do projeto, facilitando a escrita em conjunto. Também é possível visualizar a navegação por sumário e exportação do documento em formato PDF, concentrando em uma única plataforma os principais recursos necessários para a produção de trabalhos acadêmicos.

A solução foi desenvolvida seguindo uma arquitetura de monólito modular, utilizando React no frontend e Express no backend. Para simplificar o processo de configuração do ambiente, diminuindo a dificuldade inicial em sua utilização, a infraestrutura é disponibilizada por meio de contêineres Docker, eliminando a necessidade de instalação manual de dependências e bibliotecas por parte dos usuários. Dessa forma, a ferramenta reduz o esforço inicial de configuração e permite que pesquisadores e estudantes concentrem suas atividades na produção do conteúdo acadêmico.

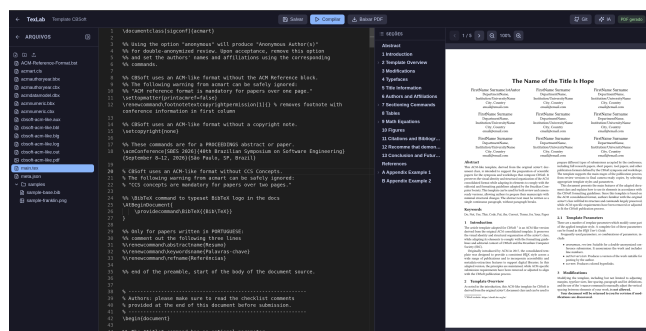


Figure 2: Layout do sistema de edição de textos em $\text{\LaTeX}$.

Durante o processo de desenvolvimento da ferramenta, foi observado que a utilização de uma LLM para realização do refinamento e criação do prompt contribuiu de forma positiva com os resultados. Ela auxiliou na robustez do prompt, consolidando a ideia, adicionando regras de negócio e requisitos não funcionais. Com isso, diminuiu a ambiguidade, inconsistências e melhorou a comunicação com o agente de desenvolvimento.

Funcionalidades simples puderam ser implementadas com um único prompt ou com poucas interações adicionais. Enquanto isso, funcionalidades mais complexas demandaram ciclos extras de desenvolvimento assistido pelo agente, sendo necessário testar a aplicação e entender onde não estava funcionando para explicar para o agente refinar um texto de melhoria.

A análise feita demonstrou a importância de um profissional técnico por trás para manusear a ferramenta durante o processo,

conduzindo o modelo a produzir código com alta qualidade. Dessa forma, a metodologia não substitui o papel do desenvolvedor, mas busca ampliar sua produtividade por meio da automação de atividades de especificação e implementação.

Por fim, observou-se que a disponibilização de documentos de contexto, como o arquivo AGENTS.md contendo informações sobre arquitetura, convenções, padrões de layout e tecnologias utilizadas, contribuiu para aumentar a aderência das implementações aos padrões previamente definidos para o projeto.

4 Conclusão

Este trabalho apresentou uma abordagem baseada em modelos de linguagem de grande escala (LLMs) e agentes inteligentes para apoiar os desenvolvedores no processo de criação de sistemas, com foco no refinamento de requisitos e na geração de artefatos de desenvolvimento. A proposta buscou explorar como a combinação de múltiplos agentes, combinada com o conhecimento humano e especializado na área de desenvolvimento de software, pode auxiliar na transformação de ideias iniciais em especificações mais estruturadas e implementáveis de forma mais fácil e rápida.

Esse texto foi escrito utilizando a ferramenta em questão, mostrando que a aplicação da metodologia foi capaz de realizar o desenvolvimento completo das principais funcionalidades de forma eficiente. Isso demonstra que o estudo e a utilização dessas novas tecnologias vieram para ajudar, quando utilizadas da forma correta.

A aplicação experimental demonstrou a sincronia entre humano e máquina, reduzindo ambiguidades nos requisitos, melhorando a organização das informações e fornecendo suporte durante diferentes etapas do ciclo de desenvolvimento. Vale ressaltar que, nesses testes, a ferramenta não foi publicada nem disponibilizada para uso geral, tendo em vista que não foram utilizados mecanismos de validação de segurança. Com isso, o sistema deve ser utilizado apenas em ambiente local para uso próprio, ficando como trabalho futuro a contribuição de como os agentes podem ser alocados para atuar como atacantes em busca de vulnerabilidades nela.

Apesar dos resultados iniciais indicarem benefícios no uso dessa abordagem, ainda existem desafios relacionados à confiabilidade nas respostas geradas, ao controle de qualidade e à definição de estratégias para validação das implementações realizadas pelos agentes. Dessa forma, o trabalho evidencia a necessidade de novos estudos buscando avaliar esses aspectos em diferentes cenários de desenvolvimento de software.

Como base para trabalhos futuros, pretende-se ampliar a avaliação experimental, analisando métricas relacionadas à qualidade dos requisitos gerados, à produtividade durante o desenvolvimento, à interação entre humanos e agentes com novas abordagens, a métricas de segurança para avaliar a confiabilidade da aplicação gerada e ao comparativo da metodologia chaining com a de multi-agentes. Além disso, novas integrações utilizando novos agentes especializados e ferramentas de engenharia de software podem ser investigadas, visando contribuir para o melhor modelo para cada implementação no processo de desenvolvimento.

Referências

- [1] Míriam B Born, Bruno C Alves, Felipe M Goulart, Leticia B Caldas, and Marilton S de Aguiar. 2025. Revisão Sistemática da Literatura integrando Sistemas Multiagente e Large Language Models. In *Workshop-Escola de Sistemas de Agentes, seus*

- Ambientes e Aplicações (WESAAC)*. SBC, 230–239.
- [2] Gabriela H Ciudrowski, Jessica I Pegorini, Francisco CM Souza, and Alinne CC Souza. 2026. Explorando uma abordagem pedagógica mediada por LLMs para o ensino de Engenharia de Requisitos: um relato de experiência. In *Workshop sobre Educação em Computação (WEI)*. SBC, 746–757.
 - [3] Ricardo Pupo Larguesa. 2025. *Engenharia de prompt para DEVS: um guia para aprender a usar a IA antes que a IA aprenda a usar você*. Casa do Código.
 - [4] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large Language Models: A Survey. doi:10.48550/ARXIV.2402.06196
 - [5] Jefferson Rodrigues do Nascimento. 2024. Exploração de técnicas de engenharia de prompt para aprimorar os resultados do uso de LLM no TCMRio. (2024).
 - [6] Varun Nair, Elliot Schumacher, Geoffrey Tso, and Anitha Kannan. 2023. DERA: Enhancing Large Language Model Completions with Dialog-Enabled Resolving Agents. doi:10.48550/ARXIV.2303.17071
 - [7] Nicolás Miccio Palermo. 2026. Agentes basados en LLMs para la derivación de tareas de desarrollo backend a partir de requerimientos textuales. In *Anais do XXIX Congresso Ibero-Americano em Engenharia de Software (CibSE 2026)* (Brasil, 2026-05-11). Sociedade Brasileira de Computação, 349–356. doi:10.5753/cibse.2026.42459
 - [8] Alexandre Pires, Enzo Kleinpaul, Lorenzo Campos, and Rafael Parizi. 2025. From Prompts to Prototypes: Exploring ChatGPT in Collaborative Software Prototyping Education. In *Anais do XXIV Simpósio Brasileiro de Qualidade de Software (SBQS 2025)* (2025-11-04). Sociedade Brasileira de Computação - SBC, 453–462. doi:10.5753/sbqs.2025.13937
 - [9] Juliana Gomes Ribeiro, Joel Machado Pires, Jonas Oliveira Pereira, Frederico Araujo Durão, and Célia Ghedini Ralha. 2025. Software Development Using a Multi-agent Approach. In *Anais do XIX Workshop-Escola de Sistemas de Agentes, seus Ambientes e Aplicações (WESAAC 2025)* (Brasil, 2025-09-29). Sociedade Brasileira de Computação, 93–100. doi:10.5753/wesaac.2025.37549
 - [10] William Jeferson Luis da Silva. 2024. ENGENHARIA DE PROMPT: Uma análise das "alucinações" em Inteligências Artificiais Generativas. (2024). <http://repositorio.unis.edu.br/handle/prefix/2727>
 - [11] Ian Sommerville. [n. d.]. *Engenharia de Software*. Pearson Universidades.
 - [12] Antônio Sergio Tolio, Enzo Zanini Becker, Felipe Colpo Bagesteiro, Cristiano Tolfo, and Lisandra Manzoni Fontoura. 2026. LLMs in Practice: Results from a Survey with Software Development Professionals in Brazil. In *Anais do XXII Simpósio Brasileiro de Sistemas de Informação (SBSI 2026)* (Brasil, 2026-05-25). Sociedade Brasileira de Computação, 637–654. doi:10.5753/sbsi.2026.248579